TEAM TOUGYPT

A judge that catches meaning changes, on any criterion

The problem: a chatbot reply can sound fine and still promise more than the policy allows.

Customer: “Can I keep the car a few days longer?”

A good — You can usually extend your rental in the app or by phone, as long as the car is not reserved for another customer.

B bad — You can usually extend your rental in the app or by phone.

B drops a condition, so it over-promises. The starting judge says both are fine: it catches only 7% of dropped conditions.

0.754 → 0.835
original criterion (combined, hold-out)

0.586 → 0.621
surprise criterion, same code

1.000
same verdict on two runs

The rules: the judge sees one reply at a time, keeps no state between replies, makes at most 3 LLM calls per reply, and nothing criterion-specific is hard-coded. At 14:30 a new criterion arrived, and the same code had to work on it.

DIAGNOSE

Step 1: where the starting judge fails, and why it matters

diagnose.py on the starting judge (226 dev cases, 5 languages)

Dropped condition	**93% missed**
“usually” → “always”	**50% missed**
Wrong number	**20% missed**
Harmless reorder	**27% wrongly rejected**
Harmless filler	**20% wrongly rejected**
Translation	**17% wrongly rejected**

Same pattern in every language: the problem is the method, not the language.

Insight 1: false alarms cost as much as misses

score.py only counts a catch if the ORIGINAL reply passes. A judge that is too strict loses catch AND keep, so precision on good replies is half the job.

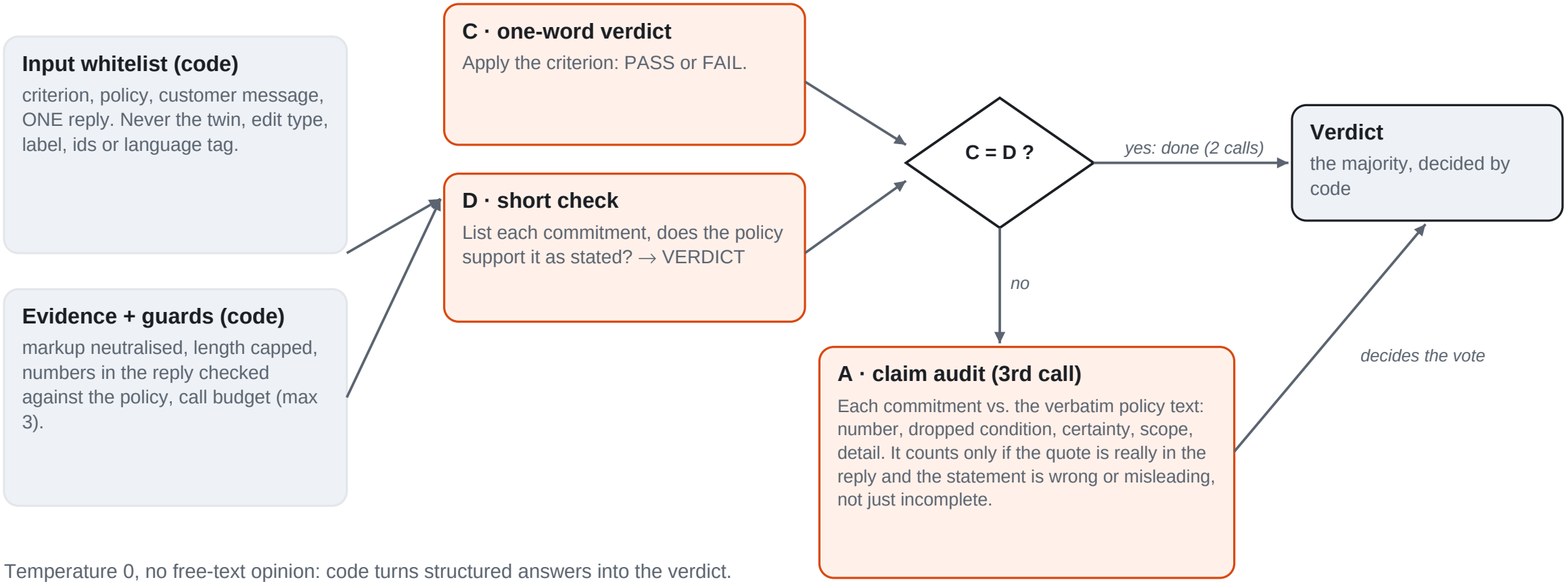
Insight 2: solve the idea, not the list

Hidden cases contain edit types we never saw. So no rule may mention an edit type, a word like “always”, a policy number or a language: behaviour must come from the criterion text, the policy and the reply alone.

We measured every change on a grouped hold-out: cases from source replies never shown to the judge.

SPLIT CODE AND LLM

Steps 2–3: how our judge works



Temperature 0, no free-text opinion: code turns structured answers into the verdict. Every signal is logged (JSONL) to re-tune offline.

STEP 4 · ITERATE AND VERIFY

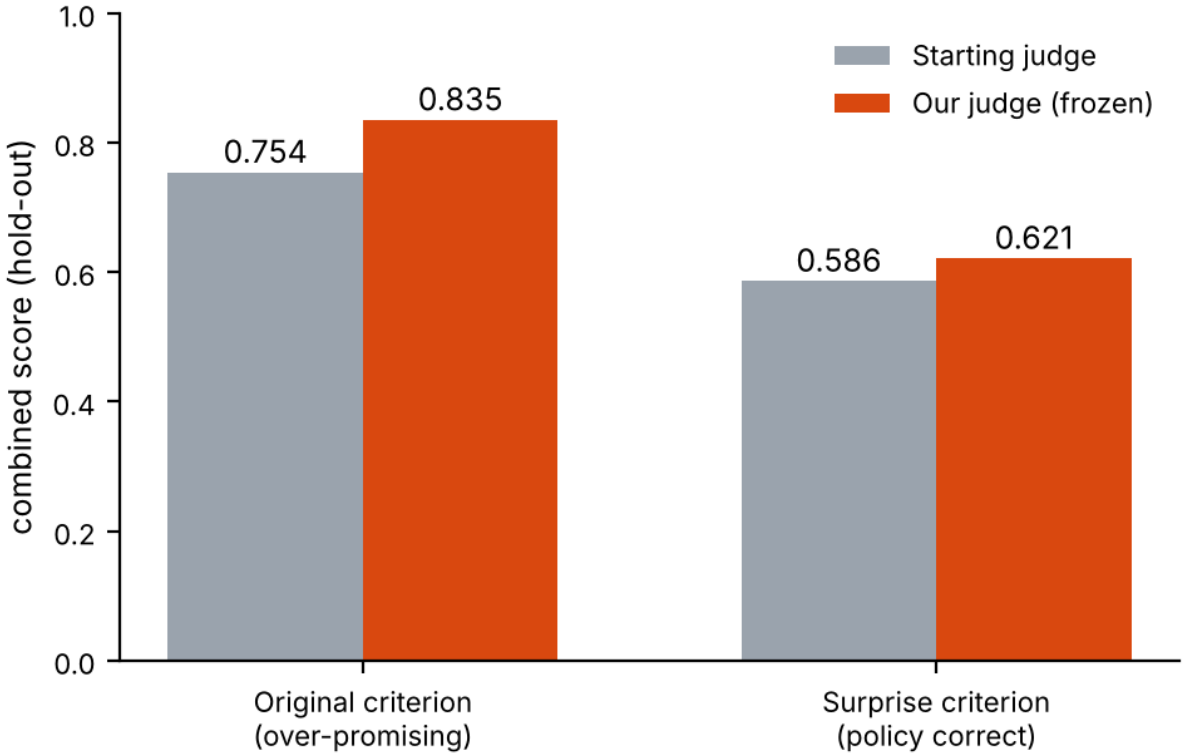
How we got there: every idea measured on both criteria

version	crit. 1	surprise	what we learned
V1 Claim audit + verify, with example pairs	–	0.401	rejected 58% of good replies
V2 + “incomplete but accurate is not an error”	–	0.350	still too strict
V3 Anchored verify / overall gate, examples	–	0.376	examples made haiku hunt for differences
Same, examples removed	0.686	0.244	too lenient on criterion 1
C One-word verdict, no examples	**0.789**	**0.596**	first to beat baseline on BOTH
A Claim audit alone	0.816	0.241	best on crit. 1, collapses on surprise
D Short reasoning alone	0.716	0.674	best on surprise, weak on crit. 1
M Majority of C, D, A (frozen)	**0.835**	**0.621**	each view fixes another's blind spot

Baseline on the same hold-out: 0.754 / 0.586. Decision rule fixed in advance: keep a change only if it wins on both criteria.

OUTCOME

Result and lessons



	Start	Ours
Original: combined	0.754	0.835
catch	0.656	0.789
kappa	0.652	0.746
Surprise: combined	0.586	0.621
Consistency (2 runs)		1.000
LLM calls per reply	1	2 (max 3)

Lessons

- A rule in a prompt is a hypothesis: measure it on criteria you did not design for.
- Few-shot pairs made the model over-strict; no examples worked better.
- No single prompt won both criteria; a vote of different views did.
- Next: the exact 3-way vote (0.882 / 0.642 offline) within budget.